

TaskForge AI Studio

Step-by-Step Setup Guide — from a blank MacBook to AI teammates writing real code on your own Kanban board

Companion to the full source code zip. Start at Step 1 and work in order — later steps assume earlier ones are done.

TaskForge AI Studio — Step-by-Step Implementation Guide

Written for macOS (MacBook Air M5 / Apple Silicon). Every command below has been written to be copy-pasteable. Follow in order — later steps assume earlier ones are done.

STEP 1 — Install Java 21

```
# Install Homebrew first if you don't have it:
curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh | /bin/bash

# Install Java 21 (Temurin/OpenJDK build)
brew install openjdk@21

# Link it so the system finds it
sudo ln -sf /opt/homebrew/opt/openjdk@21/libexec/openjdk.jdk \
/Library/Java/JavaVirtualMachines/openjdk-21.jdk

# Add to your shell profile (~/.zshrc)
echo 'export JAVA_HOME=/opt/homebrew/opt/openjdk@21' >> ~/.zshrc
echo 'export PATH="$JAVA_HOME/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

# Verify
java -version
# should print: openjdk version "21.x.x"
```

If you ever need multiple Java versions side by side later, switch to **SDKMAN** (`curl -s "https://get.sdkman.io" | bash`, then `sdk install java 21.0.4-tem`) — not required for this project, but worth knowing for interviews.

STEP 2 — Install IntelliJ IDEA Community

```
brew install --cask intellij-idea-ce
```

On first launch:

- 1 Skip the import-settings screen (new install).
- 2 Settings → Plugins → confirm **Lombok** and **Spring** plugins are enabled (bundled by default in recent versions).
- 3 Settings → Build Tools → Maven → confirm it points at a JDK 21 you'll select per-project.

STEP 3 — Install PostgreSQL

```

brew install postgresql@16
brew services start postgresql@16

# Add to PATH
echo 'export PATH="/opt/homebrew/opt/postgresql@16/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

# Create the dev database and a dedicated app user (don't use the postgres superuser in app
  config)
createdb taskforge_dev
psql taskforge_dev -c "CREATE USER taskforge WITH PASSWORD 'taskforge_dev_pw';"
psql taskforge_dev -c "GRANT ALL PRIVILEGES ON DATABASE taskforge_dev TO taskforge;"
psql taskforge_dev -c "ALTER DATABASE taskforge_dev OWNER TO taskforge;"

# Verify
psql -U taskforge -d taskforge_dev -h localhost -c "SELECT version();"

```

(Later, in Step 12, you'll have the option to run Postgres via Docker Compose instead — useful for matching teammates'/CI environments exactly. Keep this native install for now; it's faster for local iteration.)

STEP 4 — Create the Spring Boot project

Use Spring Initializr from the terminal so the setup is reproducible (no manual web-form clicking to forget later):

```

mkdir -p ~/dev/taskforge-backend && cd ~/dev/taskforge-backend

curl https://start.spring.io/starter.zip \
  -d type=maven-project \
  -d language=java \
  -d bootVersion=3.3.4 \
  -d baseDir=. \
  -d groupId=com.taskforge \
  -d artifactId=taskforge-backend \
  -d name=taskforge-backend \
  -d packageName=com.taskforge \
  -d javaVersion=21 \
  -d dependencies=web,data-jpa,postgresql,validation,security,flyway,websocket \
  -o starter.zip

unzip starter.zip && rm starter.zip

```

Add the remaining dependencies Initializr doesn't list by default — open `pom.xml` and add inside `<dependencies>`:

```

<!-- JWT -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>

<!-- API docs -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>

<!-- Lombok (optional, cuts boilerplate) -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>

<!-- Testing -->
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>postgresql</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.testcontainers</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>

```

Open the project root in IntelliJ (File → open), let Maven import finish, then create `src/main/resources/application-dev.yml`:

```

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/taskforge_dev
    username: taskforge
    password: taskforge_dev_pw
  jpa:
    hibernate:
      ddl-auto: validate # Flyway owns the schema, Hibernate never auto-generates it
    properties:
      hibernate.format_sql: true
  flyway:
    enabled: true
    locations: classpath:db/migration

server:
  port: 8080

taskforge:
  jwt:
    secret: ${JWT_SECRET:dev-only-change-me-32-chars-minimum-please}
    access-token-minutes: 15
    refresh-token-days: 7
  ollama:
    base-url: http://localhost:11434
    model: llama3.1:8b

```

And application.yml:

```

spring:
  application:
    name: taskforge-backend
  profiles:
    active: dev

```

Run it once just to confirm the skeleton boots (it'll fail at this point because there's no schema yet — that's expected, fix it in Step 5):

```
./mvnw spring-boot:run
```

STEP 5 — Create the database schema (Flyway)

Create `src/main/resources/db/migration/V1__init_schema.sql`:

```

-- Extension for UUID generation
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    avatar_url VARCHAR(500),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE workspaces (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    slug VARCHAR(100) NOT NULL UNIQUE,
    owner_id UUID NOT NULL REFERENCES users(id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE workspace_members (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    role VARCHAR(20) NOT NULL CHECK (role IN ('OWNER', 'ADMIN', 'MEMBER', 'GUEST')),
    joined_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (workspace_id, user_id)
);

CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    workspace_id UUID NOT NULL REFERENCES workspaces(id) ON DELETE CASCADE,
    key VARCHAR(10) NOT NULL,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    type VARCHAR(10) NOT NULL CHECK (type IN ('SCRUM', 'KANBAN')),
    next_issue_number INT NOT NULL DEFAULT 1,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    archived_at TIMESTAMPTZ,
    UNIQUE (workspace_id, key)
);

CREATE TABLE project_members (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    is_ai BOOLEAN NOT NULL DEFAULT FALSE,
    ai_agent_type VARCHAR(30),
    role VARCHAR(20) NOT NULL CHECK (role IN
    ('PROJECT_OWNER', 'DEVELOPER', 'TESTER', 'AI_AGENT', 'VIEWER')),

```

```

        added_at TIMESTAMPTZ NOT NULL DEFAULT now(),
        CHECK ( (is_ai = FALSE AND user_id IS NOT NULL) OR (is_ai = TRUE AND ai_agent_type IS NOT
        NULL) )
    );

CREATE TABLE workflow_statuses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    name VARCHAR(50) NOT NULL,
    category VARCHAR(15) NOT NULL CHECK (category IN ('TODO','IN_PROGRESS','DONE')),
    position INT NOT NULL,
    color VARCHAR(10)
);

CREATE TABLE sprints (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    goal TEXT,
    start_date DATE,
    end_date DATE,
    status VARCHAR(15) NOT NULL DEFAULT 'PLANNED' CHECK (status IN
    ('PLANNED','ACTIVE','COMPLETED'))
);

CREATE TABLE issues (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    issue_key VARCHAR(20) NOT NULL,
    type VARCHAR(10) NOT NULL CHECK (type IN ('EPIC','STORY','TASK','SUBTASK','BUG')),
    parent_issue_id UUID REFERENCES issues(id),
    epic_id UUID REFERENCES issues(id),
    sprint_id UUID REFERENCES sprints(id),
    status_id UUID NOT NULL REFERENCES workflow_statuses(id),
    title VARCHAR(500) NOT NULL,
    description TEXT,
    priority VARCHAR(10) NOT NULL DEFAULT 'MEDIUM' CHECK (priority IN
    ('LOWEST','LOW','MEDIUM','HIGH','HIGHEST')),
    story_points INT,
    assignee_id UUID REFERENCES users(id),
    assignee_is_ai BOOLEAN NOT NULL DEFAULT FALSE,
    assignee_ai_type VARCHAR(30),
    reporter_id UUID NOT NULL REFERENCES users(id),
    due_date DATE,
    board_position INT NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    resolved_at TIMESTAMPTZ,
    deleted_at TIMESTAMPTZ,
    UNIQUE (project_id, issue_key)
);

```

```

);

CREATE TABLE labels (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  name VARCHAR(50) NOT NULL,
  color VARCHAR(10) NOT NULL
);

CREATE TABLE issue_labels (
  issue_id UUID NOT NULL REFERENCES issues(id) ON DELETE CASCADE,
  label_id UUID NOT NULL REFERENCES labels(id) ON DELETE CASCADE,
  PRIMARY KEY (issue_id, label_id)
);

CREATE TABLE comments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  issue_id UUID NOT NULL REFERENCES issues(id) ON DELETE CASCADE,
  author_id UUID REFERENCES users(id),
  author_is_ai BOOLEAN NOT NULL DEFAULT FALSE,
  author_ai_type VARCHAR(30),
  body TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ,
  deleted_at TIMESTAMPTZ
);

CREATE TABLE mentions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  comment_id UUID NOT NULL REFERENCES comments(id) ON DELETE CASCADE,
  mentioned_user_id UUID NOT NULL REFERENCES users(id)
);

CREATE TABLE activity_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  issue_id UUID REFERENCES issues(id),
  actor_id UUID REFERENCES users(id),
  actor_is_ai BOOLEAN NOT NULL DEFAULT FALSE,
  actor_ai_type VARCHAR(30),
  action_type VARCHAR(50) NOT NULL,
  field_name VARCHAR(50),
  old_value TEXT,
  new_value TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

```



```

        type VARCHAR(40) NOT NULL,
        title VARCHAR(255) NOT NULL,
        body TEXT,
        link VARCHAR(500),
        actor_id UUID,
        actor_is_ai BOOLEAN NOT NULL DEFAULT FALSE,
        is_read BOOLEAN NOT NULL DEFAULT FALSE,
        created_at TIMESTAMPTZ NOT NULL DEFAULT now()
    );

CREATE TABLE ai_agent_settings (
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    agent_type VARCHAR(30) NOT NULL,
    enabled BOOLEAN NOT NULL DEFAULT TRUE,
    autonomy_level VARCHAR(15) NOT NULL DEFAULT 'SEMI_AUTO' CHECK (autonomy_level IN
    ('MANUAL', 'SEMI_AUTO', 'AUTO')),
    model_override VARCHAR(50),
    PRIMARY KEY (project_id, agent_type)
);

CREATE TABLE ai_agent_executions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    issue_id UUID NOT NULL REFERENCES issues(id) ON DELETE CASCADE,
    agent_type VARCHAR(30) NOT NULL,
    trigger_event VARCHAR(50) NOT NULL,
    status VARCHAR(15) NOT NULL DEFAULT 'PENDING' CHECK (status IN
    ('PENDING', 'RUNNING', 'AWAITING_REVIEW', 'COMPLETED', 'FAILED')),
    input_context JSONB,
    output_result JSONB,
    model_used VARCHAR(50),
    tokens_used INT,
    started_at TIMESTAMPTZ,
    completed_at TIMESTAMPTZ,
    error_message TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE ai_agent_suggestions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    execution_id UUID NOT NULL REFERENCES ai_agent_executions(id) ON DELETE CASCADE,
    issue_id UUID NOT NULL REFERENCES issues(id) ON DELETE CASCADE,
    agent_type VARCHAR(30) NOT NULL,
    suggestion_type VARCHAR(30) NOT NULL,
    content JSONB NOT NULL,
    status VARCHAR(15) NOT NULL DEFAULT 'PENDING' CHECK (status IN
    ('PENDING', 'ACCEPTED', 'REJECTED', 'AUTO_APPLIED')),
    reviewed_by UUID REFERENCES users(id),
    reviewed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

```

CREATE TABLE ai_task_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  issue_id UUID NOT NULL REFERENCES issues(id) ON DELETE CASCADE,
  agent_type VARCHAR(30) NOT NULL,
  trigger_event VARCHAR(50) NOT NULL,
  priority INT NOT NULL DEFAULT 0,
  status VARCHAR(15) NOT NULL DEFAULT 'QUEUED' CHECK (status IN
('QUEUED', 'PROCESSING', 'DONE', 'FAILED')),
  retry_count INT NOT NULL DEFAULT 0,
  enqueued_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  started_at TIMESTAMPTZ,
  finished_at TIMESTAMPTZ
);

-- Indexes that matter from day one
CREATE INDEX idx_issues_project ON issues(project_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_issues_assignee ON issues(assignee_id);
CREATE INDEX idx_issues_sprint ON issues(sprint_id);
CREATE INDEX idx_activity_issue ON activity_logs(issue_id);
CREATE INDEX idx_notifications_user_unread ON notifications(user_id) WHERE is_read = FALSE;
CREATE INDEX idx_ai_queue_status ON ai_task_queue(status);

```

Run the app again — Flyway applies this automatically on startup (`./mvnw spring-boot:run`). Confirm with:

```
psql -U taskforge -d taskforge_dev -h localhost -c "\dt"
```

You should see all the tables, plus Flyway's own `flyway_schema_history`.

STEP 6 — Create the authentication system

6.1 User entity

```
// src/main/java/com/taskforge/identity/domain/User.java
package com.taskforge.identity.domain;

import jakarta.persistence.*;
import org.hibernate.annotations.UuidGenerator;
import java.time.Instant;
import java.util.UUID;

@Entity
@Table(name = "users")
public class User {

    @Id
    @UuidGenerator
    private UUID id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(name = "password_hash", nullable = false)
    private String passwordHash;

    @Column(name = "full_name", nullable = false)
    private String fullName;

    @Column(name = "avatar_url")
    private String avatarUrl;

    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt = Instant.now();

    protected User() {}

    public User(String email, String passwordHash, String fullName) {
        this.email = email;
        this.passwordHash = passwordHash;
        this.fullName = fullName;
    }

    public UUID getId() { return id; }
    public String getEmail() { return email; }
    public String getPasswordHash() { return passwordHash; }
    public String getFullName() { return fullName; }
}
```

6.2 Repository

```
// identity/infrastructure/UserRepository.java
package com.taskforge.identity.infrastructure;

import com.taskforge.identity.domain.User;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;
import java.util.UUID;

public interface UserRepository extends JpaRepository<User, UUID> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

6.3 JWT service

```

// identity/application/JwtService.java
package com.taskforce.identity.application;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import javax.crypto.SecretKey;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.UUID;

@Service
public class JwtService {

    private final SecretKey key;
    private final long accessTokenMinutes;

    public JwtService(
        @Value("${taskforce.jwt.secret}") String secret,
        @Value("${taskforce.jwt.access-token-minutes}") long accessTokenMinutes) {
        this.key = Keys.hmacShaKeyFor(secret.getBytes());
        this.accessTokenMinutes = accessTokenMinutes;
    }

    public String generateAccessToken(UUID userId, String email) {
        Instant now = Instant.now();
        return Jwts.builder()
            .subject(userId.toString())
            .claim("email", email)
            .issuedAt(java.util.Date.from(now))
            .expiration(java.util.Date.from(now.plus(accessTokenMinutes,
ChronoUnit.MINUTES)))
            .signWith(key)
            .compact();
    }

    public UUID extractUserId(String token) {
        String subject = Jwts.parser().verifyWith(key).build()
            .parseSignedClaims(token).getPayload().getSubject();
        return UUID.fromString(subject);
    }

    public boolean isValid(String token) {
        try {
            Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
            return true;
        } catch (Exception e) {
            return false;
        }
    }

}

```

6.4 Security config (Spring Security 6 style — no `WebSecurityConfigurerAdapter`)

```

// config/SecurityConfig.java
package com.taskforge.config;

import com.taskforge.identity.application.JwtAuthFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http, JwtAuthFilter jwtAuthFilter)
        throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // stateless JWT API, no cookies for auth state
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**", "/swagger-ui/**", "/v3/api-
docs/**").permitAll()
                .requestMatchers("/ws/**").permitAll() // WebSocket handshake auth handled
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

6.5 JWT filter

```
// identity/application/JwtAuthFilter.java
package com.taskforge.identity.application;

import jakarta.servlet.FilterChain;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
import java.io.IOException;
import java.util.List;

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    public JwtAuthFilter(JwtService jwtService) { this.jwtService = jwtService; }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws java.io.IOException, jakarta.servlet.ServletException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtService.isValid(token)) {
                var userId = jwtService.extractUserId(token);
                var auth = new UsernamePasswordAuthenticationToken(userId, null, List.of());
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

6.6 Auth controller

```
// identity/web/AuthController.java
package com.taskforge.identity.web;

import com.taskforge.identity.application.AuthService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/auth")
public class AuthController {

    private final AuthService authService;
    public AuthController(AuthService authService) { this.authService = authService; }

    public record RegisterRequest(String email, String password, String fullName) {}
    public record LoginRequest(String email, String password) {}
    public record AuthResponse(String accessToken, String userId, String fullName) {}

    @PostMapping("/register")
    public AuthResponse register(@RequestBody RegisterRequest req) {
        return authService.register(req.email(), req.password(), req.fullName());
    }

    @PostMapping("/login")
    public AuthResponse login(@RequestBody LoginRequest req) {
        return authService.login(req.email(), req.password());
    }
}
```

```
// identity/application/AuthService.java
package com.taskforce.identity.application;

import com.taskforce.identity.domain.User;
import com.taskforce.identity.infrastructure.UserRepository;
import com.taskforce.identity.web.AuthController.AuthResponse;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
public class AuthService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;

    public AuthService(UserRepository userRepository, PasswordEncoder passwordEncoder,
        JwtService jwtService) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
    }

    public AuthResponse register(String email, String rawPassword, String fullName) {
        if (userRepository.existsByEmail(email)) {
            throw new IllegalArgumentException("Email already registered");
        }
        User user = new User(email, passwordEncoder.encode(rawPassword), fullName);
        userRepository.save(user);
        String token = jwtService.generateAccessToken(user.getId(), email);
        return new AuthResponse(token, user.getId().toString(), fullName);
    }

    public AuthResponse login(String email, String rawPassword) {
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new IllegalArgumentException("Invalid credentials"));
        if (!passwordEncoder.matches(rawPassword, user.getPasswordHash())) {
            throw new IllegalArgumentException("Invalid credentials");
        }
        String token = jwtService.generateAccessToken(user.getId(), email);
        return new AuthResponse(token, user.getId().toString(), user.getFullName());
    }
}
```

Test it:

```
curl -X POST http://localhost:8080/api/v1/auth/register \
-H "Content-Type: application/json" \
-d '{"email":"you@example.com","password":"SecurePass123!","fullName":"You"}'

curl -X POST http://localhost:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"you@example.com","password":"SecurePass123!"}'
```

You now have working JWT auth. **Refresh tokens and the `@PreAuthorize` permission evaluator from §10 of the Architecture Spec** are the natural next thing to add once **Workspaces/Projects** exist — they need the `project_members`/`workspace_members` tables to check roles against.

STEP 7 — Frontend setup


```
cd ~/dev
npm create vite@latest taskforge-frontend -- --template react-ts
cd taskforge-frontend
npm install

npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

npm install @tanstack/react-query zustand react-router-dom axios \
  react-hook-form zod @hookform/resolvers date-fns @dnd-kit/core @dnd-kit/sortable \
  recharts lucide-react
```

Add shadcn/ui (generates `components.json` + copies primitives into your repo — you own the code, no runtime dependency):

```
npx shadcn@latest init
npx shadcn@latest add button card badge avatar dialog dropdown-menu input select
```

Configure `tailwind.config.ts` content to scan `./src/**/*.{ts,tsx}`, and drop the design tokens from Architecture Spec §6.4 into `src/index.css` as CSS variables (`--color-bg`, `--color-primary`, `--color-accent-ai`, etc.) so both the dark and light themes are driven from one source of truth.

Run it: `npm run dev` → `http://localhost:5173`.

STEP 8 — Install Ollama and pull a model (this powers the in-app AI teammates)

```
brew install ollama
brew services start ollama

# Pull a model that runs comfortably on Apple Silicon 16GB+
ollama pull llama3.1:8b

# Quick sanity check
ollama run llama3.1:8b "Say hello in 5 words"
```

Confirm the REST API is up (Spring will call this, not the CLI):

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.1:8b",
  "prompt": "Reply with just the word OK",
  "stream": false
}'
```

If your Mac has less RAM, use `mistral:7b` or `phi3:mini` instead — same API, just change `taskforge.ollama.model` in `application-dev.yml`.

STEP 9 — Build the AI agent module (the part that makes AI teammates real)

9.1 Core contracts

```
// aiagent/domain/AgentType.java
package com.taskforge.aiagent.domain;

public enum AgentType {
    ARCHITECT_AI, CODE_AI, TESTER_AI, REVIEWER_AI, DOCUMENTATION_AI, PROJECT_MANAGER_AI
}
```

```
// aiagent/domain/AIAgent.java
package com.taskforge.aiagent.domain;

public interface AIAgent {
    AgentType type();
    AgentResult execute(AgentContext context);
}
```

```
// aiagent/domain/AgentContext.java
package com.taskforge.aiagent.domain;

import java.util.List;
import java.util.UUID;

public record AgentContext(
    UUID issueId,
    String issueTitle,
    String issueDescription,
    String triggerEvent
) {}
```

```
// aiagent/domain/AgentResult.java
package com.taskforge.aiagent.domain;

import java.util.List;

public record AgentResult(
    AgentType agentType,
    List<SubIssueDraft> subIssues,
    String commentMarkdown,
    String rawJsonPayload,
    String modelUsed
) {
    public record SubIssueDraft(String title, String description) {}
}
```

9.2 LLM client (pluggable — Ollama today, anything else later without touching agent code)

```
// aiagent/llm/LLMClient.java
package com.taskforge.aiagent.llm;

public interface LLMClient {
    String completeJson(String systemPrompt, String userPrompt);
}
```

```

// aiagent/llm/OllamaLLMClient.java
package com.taskforge.aiagent.llm;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;
import java.util.Map;

@Component
public class OllamaLLMClient implements LLMClient {

    private final RestClient restClient;
    private final String model;
    private final ObjectMapper mapper = new ObjectMapper();

    public OllamaLLMClient(
        @Value("${taskforge.ollama.base-url}") String baseUrl,
        @Value("${taskforge.ollama.model}") String model) {
        this.restClient = RestClient.create(baseUrl);
        this.model = model;
    }

    @Override
    public String completeJson(String systemPrompt, String userPrompt) {
        Map<String, Object> body = Map.of(
            "model", model,
            "system", systemPrompt,
            "prompt", userPrompt,
            "format", "json", // ask Ollama to constrain output to valid JSON
            "stream", false
        );

        String raw = restClient.post()
            .uri("/api/generate")
            .body(body)
            .retrieve()
            .body(String.class);

        try {
            JsonNode node = mapper.readTree(raw);
            return node.get("response").asText(); // the model's JSON, as a string
        } catch (Exception e) {
            throw new RuntimeException("Failed to parse Ollama response", e);
        }
    }
}

```

9.3 ArchitectAI — the first agent, end to end

```

// aiagent/agents/ArchitectAIAgent.java
package com.taskforge.aiagent.agents;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.taskforge.aiagent.domain.*;
import com.taskforge.aiagent.llm.LLMClient;
import org.springframework.stereotype.Component;
import java.util.ArrayList;
import java.util.List;

@Component
public class ArchitectAIAgent implements AIAgent {

    private static final String SYSTEM_PROMPT = ""
        You are ArchitectAI, a software architecture assistant embedded in a project
        management tool. Given an epic or large task, propose 3 to 7 concrete,
        actionable subtasks and a short architecture note.
        Respond ONLY with JSON, no prose, matching exactly this shape:
        {"subtasks": [{"title": "string", "description": "string"}], "architectureNotes":
"string"}
        """;

    private final LLMClient llmClient;
    private final ObjectMapper mapper = new ObjectMapper();

    public ArchitectAIAgent(LLMClient llmClient) { this.llmClient = llmClient; }

    @Override
    public AgentType type() { return AgentType.ARCHITECT_AI; }

    @Override
    public AgentResult execute(AgentContext context) {
        String userPrompt = ""
            Epic title: %s
            Epic description: %s
            """.formatted(context.issueTitle(), context.issueDescription());

        String jsonResponse = llmClient.completeJson(SYSTEM_PROMPT, userPrompt);

        try {
            JsonNode root = mapper.readTree(jsonResponse);
            List<AgentResult.SubIssueDraft> subtasks = new ArrayList<>();
            for (JsonNode node : root.get("subtasks")) {
                subtasks.add(new AgentResult.SubIssueDraft(
                    node.get("title").asText(),
                    node.get("description").asText()
                ));
            }
            String notes = root.get("architectureNotes").asText();

            String comment = "***ArchitectAI proposal:**\n\n" + notes;

            return new AgentResult(AgentType.ARCHITECT_AI, subtasks, comment, jsonResponse,
"llama3.1:8b");
        } catch (Exception e) {
            // Never let a malformed LLM response crash the pipeline – fail safe, log, retry
            later
            return new AgentResult(AgentType.ARCHITECT_AI, List.of(),
                "ArchitectAI could not parse a valid response and will retry.",
                jsonResponse, "llama3.1:8b");
        }
    }
}

```

9.4 Orchestrator — listens for issue creation, decides which agents fire

```
// aiagent/orchestration/IssueCreatedEvent.java
package com.taskforge.aiagent.orchestration;

import java.util.UUID;

public record IssueCreatedEvent(UUID issueId, String issueType) {}
```

```
// aiagent/orchestration/AIOrchestrator.java
package com.taskforge.aiagent.orchestration;

import com.taskforge.aiagent.domain.AgentType;
import com.taskforge.aiagent.infrastructure.AITaskQueueRepository;
import com.taskforge.aiagent.infrastructure.AITaskQueueEntity;
import org.springframework.context.event.EventListener;
import org.springframework.stereotype.Component;
import java.util.List;

@Component
public class AIOrchestrator {

    private final AITaskQueueRepository queueRepository;

    public AIOrchestrator(AITaskQueueRepository queueRepository) {
        this.queueRepository = queueRepository;
    }

    @EventListener
    public void onIssueCreated(IssueCreatedEvent event) {
        List<AgentType> agentsToTrigger = switch (event.issueType()) {
            case "EPIC" -> List.of(AgentType.ARCHITECT_AI, AgentType.PROJECT_MANAGER_AI);
            case "STORY", "TASK" -> List.of(AgentType.PROJECT_MANAGER_AI);
            default -> List.of();
        };

        for (AgentType agentType : agentsToTrigger) {
            queueRepository.save(AITaskQueueEntity.queued(event.issueId(), agentType,
                "ISSUE_CREATED"));
        }
    }
}
```

9.5 Worker — actually executes queued agent tasks (this is the "AI acts on its own" loop)

```

// aiagent/orchestration/AITaskQueueWorker.java
package com.taskforge.aiagent.orchestration;

import com.taskforge.aiagent.domain.AIAgent;
import com.taskforge.aiagent.domain.AgentContext;
import com.taskforge.aiagent.domain.AgentResult;
import com.taskforge.aiagent.domain.AgentType;
import com.taskforge.aiagent.infrastructure.AITaskQueueEntity;
import com.taskforge.aiagent.infrastructure.AITaskQueueRepository;
import com.taskforge.issue.application.IssueService;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import java.util.List;
import java.util.Map;

@Component
public class AITaskQueueWorker {

    private final AITaskQueueRepository queueRepository;
    private final IssueService issueService; // existing service used by humans too – same code path
    private final Map<AgentType, AIAgent> agentsByType;

    public AITaskQueueWorker(AITaskQueueRepository queueRepository, IssueService issueService,
        List<AIAgent> agents) {
        this.queueRepository = queueRepository;
        this.issueService = issueService;
        this.agentsByType = agents.stream()
            .collect(java.util.stream.Collectors.toMap(AIAgent::type, a -> a));
    }

    @Scheduled(fixedDelay = 5000) // poll every 5s – simple and good enough for Phase 1
    public void processQueue() {
        List<AITaskQueueEntity> queued =
            queueRepository.findTop10ByStatusOrderByEnqueuedAtAsc("QUEUED");

        for (AITaskQueueEntity task : queued) {
            task.markProcessing();
            queueRepository.save(task);

            try {
                AIAgent agent = agentsByType.get(task.getAgentType());
                var issue = issueService.getById(task.getIssueId());

                AgentContext context = new AgentContext(
                    issue.getId(), issue.getTitle(), issue.getDescription(),
                    task.getTriggerEvent());

                AgentResult result = agent.execute(context);
            }
        }
    }
}

```

```

        // Same service path a human controller would use – AI gets no special back door
        result.subIssues().forEach(draft ->
            issueService.createSubIssue(issue.getId(), draft.title(),
draft.description(),
            true, task.getAgentType()));

        if (result.commentMarkdown() != null) {
            issueService.addComment(issue.getId(), result.commentMarkdown(), true,
task.getAgentType());
        }

        task.markDone();
    } catch (Exception e) {
        task.markFailedAndMaybeRetry();
    }
    queueRepository.save(task);
}
}
}

```

This is the complete loop: **create an Epic** → **event fires** → **orchestrator enqueues ArchitectAI and ProjectManagerAI** → **worker picks them up within 5 seconds** → **ArchitectAI calls your local Llama model** → **sub-issues and a comment appear on the epic automatically**, through the exact same `IssueService` a human action would use. Repeat the same pattern (new agent class implementing `AIAgent`, new case in the orchestrator's switch) for `CodeAI`, `TesterAI`, `ReviewerAI`, `DocumentationAI`, and `ProjectManagerAI` — they all plug into this identical pipeline.

STEP 10 — Walk the full example: "Build Multiplayer System"

- 1 `POST /api/v1/projects/{id}/issues` with `{"type":"EPIC","title":"Build Multiplayer System","description":"Real-time multiplayer for the game lobby."}`.
- 2 `IssueService.createIssue()` saves the row and publishes `IssueCreatedEvent(issueId, "EPIC")`.
- 3 `AIOrchestrator.onIssueCreated` enqueues `ARCHITECT_AI` and `PROJECT_MANAGER_AI` rows in `ai_task_queue`.
- 4 Within 5s, `AITaskQueueWorker` picks up the `ARCHITECT_AI` row, builds an `AgentContext`, calls `ArchitectAIAgent.execute()`.
- 5 `ArchitectAIAgent` prompts Ollama; the model returns JSON subtasks like `"Design network sync protocol"`, `"Implement server-authoritative state"`, `"Client-side prediction & reconciliation"`.
- 6 The worker creates those as real sub-issues under the epic and posts ArchitectAI's architecture-notes comment — through `IssueService`, identical to a human doing it via the UI.
- 7 Refresh the board in the frontend (or wait for the websocket push once you've wired it) — the new sub-issues and the AI comment are just there, attributed to **ArchitectAI**.
- 8 Repeat for `PROJECT_MANAGER_AI` in parallel — it posts a priority/risk comment on the same epic.

That sequence is your demo. Record it as a short screen capture for your resume/portfolio — it's the single most convincing thing you can show in an interview.

STEP 11 — Docker Compose for the full local stack

```
# docker-compose.yml (repo root)
version: "3.9"
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: taskforge_dev
      POSTGRES_USER: taskforge
      POSTGRES_PASSWORD: taskforge_dev_pw
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]

  redis:
    image: redis:7
    ports: ["6379:6379"]

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: taskforge
      MINIO_ROOT_PASSWORD: taskforge_dev_pw
    ports: ["9000:9000", "9001:9001"]
    volumes: ["miniodata:/data"]

  ollama:
    image: ollama/ollama
    ports: ["11434:11434"]
    volumes: ["ollamadata:/root/.ollama"]

volumes:
  pgdata:
  miniodata:
  ollamadata:
```

```
docker compose up -d
docker exec -it $(docker ps -qf "ancestor=ollama/ollama") ollama pull llama3.1:8b
```

(Redis and MinIO are only needed starting in Phase 2 — fine to leave them stopped while you're still on Phase 1.)

STEP 12 — Testing strategy

- **Unit tests** (no Spring context): test each `AI Agent` implementation with a fake `LLMClient` that returns canned JSON — verifies your parsing logic without needing Ollama running.
- **Integration tests**: Testcontainers spins up a real throwaway Postgres for `IssueService/repository` tests — never test against your dev database.
- **API tests**: `@SpringBootTest` + `MockMvc` (or `RestAssured`) hitting real endpoints with a Testcontainers Postgres.
- **Frontend**: Vitest for component/unit tests, Playwright for one true end-to-end test (create epic → see AI sub-issues appear) — this single e2e test is worth more in an interview than 50 unit tests, because it proves the whole story works.

STEP 13 — Git, GitHub, and making it resume-worthy


```
cd ~/dev/taskforge-backend && git init && git add -A && git commit -m "Initial Spring Boot skeleton"
gh repo create taskforge-ai-studio --public --source=. --push
```

Write a README with: a 10-second GIF of the Epic → ArchitectAI demo from Step 10, the architecture diagram from the Architecture Spec, and a "Why this project" section that states the human/AI collaboration thesis in one paragraph. Add a GitHub Actions workflow (.github/workflows/ci.yml) running `./mvnw test` and `npm test` on every push — free on GitHub.

STEP 14 — Free deployment for a live demo link (so the AI agents work for **anyone** who visits, not just on your Mac)

The core issue: Ollama only serves whatever machine it's running on. Your laptop's Ollama can't be "uploaded" anywhere — it's a running process, not a file. So a public site needs the LLM running **somewhere reachable**, which means one of two real options:

Option A — Self-host Ollama on a free always-on server (same setup, different machine)

Oracle Cloud's Always Free tier is the standout option for this: an ARM VM with real RAM, free forever, no time limit (unlike AWS/GCP's 12-month free trials). Historically 4 OCPUs / 24GB RAM; there are recent (June 2026) reports of Oracle trimming new free-tier signups to 2 OCPUs / 12GB — **verify the current limit when you sign up**, terms shift. Even at 12GB, an 8B quantized model runs fine (~5–8 tokens/sec).

```
# On the Oracle VM, after creating an Ampere A1 instance (Ubuntu) and SSH-ing in:
curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3.1:8b
# open port 11434 in the VM's security list/firewall, or better: keep it private
# and only let your own Spring Boot app (running on the same VM) call localhost:11434
```

Best version of this option: run your **entire** `docker-compose.yml` **stack from Step 11** (Postgres, backend, Ollama) on that one free VM. One always-free server, the whole real website, AI agents included, \$0/month.

Option B — Swap in a free hosted LLM API for the public deployment, keep Ollama for local dev

Because `LLMClient` is an interface (Step 9.2), you can add a second implementation and use it only in production — zero changes to the 6 agent classes. **Groq** is a good fit: free tier, no credit card required, OpenAI-compatible API, and it serves open models (Llama, etc.) on its own fast hardware so you're not waiting on anyone's GPU:

```
@Component
@Profile("prod")
public class GroqLLMClient implements LLMClient {
    private final RestClient restClient;
    public GroqLLMClient(@Value("${taskforge.groq.api-key}") String apiKey) {
        this.restClient = RestClient.builder()
            .baseUrl("https://api.groq.com/openai/v1")
            .defaultHeader("Authorization", "Bearer " + apiKey)
            .build();
    }
    // same completeJson(systemPrompt, userPrompt) shape — call /chat/completions
    // with model "llama-3.3-70b-versatile" instead of Ollama's /api/generate
}
```

Groq's free tier has real rate limits (roughly tens of requests/minute) — fine for a portfolio demo with a handful of visitors, not for production traffic. That's an upgrade-later problem, not a now problem.

Rest of the stack

Piece	Free option
Backend (if not using Option A's all-in-one VM)	Render or Railway free tier (Docker deploy)
Database	Neon or Supabase free Postgres, or Postgres on the same Oracle VM
Frontend	Vercel or Netlify free tier
Domain	Use the free *.vercel.app/Oracle public IP at first — a real domain isn't required to be "fully deployed"

Recommended path for you specifically: Option A (one Oracle free VM running everything via Docker Compose) gets you the closest to "a real, fully working Jira-like website with AI agents acting on their own" for an actual stranger to visit — not just a recording. Pair it with Option B's Groq swap as a fallback if the VM's CPU-only inference ever feels too slow for a live demo.

What to build next

Once Steps 1–11 work end-to-end, go back to the **Phased Roadmap** document and continue with the rest of Phase 1 (Kanban board UI, remaining 5 agents, notifications), then Phase 2. Every later feature follows the same shape you just built: an entity, a service method existing code already calls, a domain event, and — where relevant — an AIAgent that plugs into the same orchestrator/worker pair.